

ICG Online Assignment (A2)

Chain of Assignments

Time: 40 Mins Total Marks: 10

Problem

In this online, you are required to extend the grammar to support **chain of assignments**. Thus, your program should support the following syntax.

```
int a, b, c;  
a = b = c = 10; //chain of assignments
```

Task

1. Extend the current grammar to support **chain of assignments**. You need to preserve the associativity of the assignment operator which is right associative. Like the offline, you need to work only with integer type variables. There will be no syntax or semantic errors in the given input program.
2. Update your ICG code to generate assembly code for this new rule/syntax. The assembly code output by your program must correctly be assembled with 'fasm'. Subsequently, the binary produced by 'fasm' for against your assembly code must produce the expected output in the input program.

Mark Distribution

- Extending the grammar to support chain of assignments. (4)
- Code generation for chain of assignments. (4)
- Preserving associativity of the assignment operator. (2)

Find 3 (three) sample input-output in the next pages.

Sample Input-Output

- Program with local variables

Input Program	Generated Sample Assembly (print library omitted)
<pre> 1 int main(){ 2 int a, b, c; 3 a = b = c = 2*3+4; 4 println(a); 5 println(b); 6 println(c); 7 return 0; 8 } 9 </pre>	<pre> format ELF executable 3 entry main segment readable writeable segment readable executable main: PUSH EBP MOV EBP, ESP SUB ESP, 4 SUB ESP, 4 SUB ESP, 4 .L1: MOV EAX, 3 ; Line 3 MOV ECX, EAX MOV EAX, 2 ; Line 3 CWD MUL ECX PUSH EAX MOV EAX, 4 ; Line 3 MOV EDI, EAX POP EAX ; Line 3 ADD EAX, EDI MOV [EBP-12], EAX MOV [EBP-8], EAX MOV [EBP-4], EAX .L2: MOV EAX, [EBP-4] ; Line 4 CALL print_number .L3: MOV EAX, [EBP-8] ; Line 5 CALL print_number .L4: MOV EAX, [EBP-12] ; Line 6 CALL print_number .L5: MOV EAX, 0 ; Line 7 JMP .L7 .L6: .L7: ADD ESP, 12 POP EBP MOV EAX, 1 XOR EBX, EBX INT 0x80 POP EBP RET </pre>
Expected Output (from the generated binary by fasm) <pre> 10 10 10 </pre>	

- Program with global variables

Input Program	Generated Sample Assembly (print library omitted)
<pre> 1 int a, b, c; 2 int main(){ 3 a = b = c = 2*3+4; 4 println(a); 5 println(b); 6 println(c); 7 return 0; 8 } 9 </pre>	<pre> format ELF executable 3 entry main segment readable writeable a dd 1 DUP (0) b dd 1 DUP (0) c dd 1 DUP (0) segment readable executable main: PUSH EBP MOV EBP, ESP MOV EAX, 3 ; Line 3 MOV ECX, EAX MOV EAX, 2 ; Line 3 CWD MUL ECX PUSH EAX MOV EAX, 4 ; Line 3 MOV EDX, EAX POP EAX ; Line 3 ADD EAX, EDX MOV [c], EAX MOV [b], EAX MOV [a], EAX .L1: MOV EAX, [a] ; Line 4 CALL print_number .L2: MOV EAX, [b] ; Line 5 CALL print_number .L3: MOV EAX, [c] ; Line 6 CALL print_number .L4: MOV EAX, 0 ; Line 7 JMP .L6 .L5: .L6: POP EBP MOV EAX, 1 XOR EBX, EBX INT 0x80 POP EBP RET </pre>
Expected Output (from the generated binary by fasm)	
<pre> 10 10 10 </pre>	

- Program demonstrating right associativity of assignment operator

Input Program	Generated Sample Assembly (print library omitted)
<pre> 1 int a, b; 2 int main(){ 3 int x, y; 4 x = 1; 5 b = 2; 6 y = 3; 7 a = x = b = y = 2*3+9; 8 println(a); 9 println(b); 10 println(x); 11 println(y); 12 return 0; 13 } 14 </pre>	<pre> format ELF executable 3 entry main segment readable writeable a dd 1 DUP (0) b dd 1 DUP (0) segment readable executable main: PUSH EBP MOV EBP, ESP SUB ESP, 4 SUB ESP, 4 .L1: MOV EAX, 1 ; Line 4 MOV [EBP-4], EAX .L2: MOV EAX, 2 ; Line 5 MOV [b], EAX .L3: MOV EAX, 3 ; Line 6 MOV [EBP-8], EAX .L4: MOV EAX, 3 ; Line 7 MOV ECX, EAX MOV EAX, 2 ; Line 7 CWD MUL ECX PUSH EAX MOV EAX, 9 ; Line 7 MOV EDX, EAX POP EAX ; Line 7 ADD EAX, EDX MOV [EBP-8], EAX MOV [b], EAX MOV [EBP-4], EAX MOV [a], EAX .L5: MOV EAX, [a] ; Line 8 CALL print_number .L6: MOV EAX, [b] ; Line 9 CALL print_number .L7: MOV EAX, [EBP-4] ; Line 10 CALL print_number .L8: MOV EAX, [EBP-8] ; Line 11 CALL print_number .L9: MOV EAX, 0 ; Line 12 JMP .L11 .L10: .L11: ADD ESP, 8 POP EBP MOV EAX, 1 XOR EBX, EBX INT 0x80 </pre>
Expected Output (from the generated binary by fasm)	
<pre> 15 15 15 15 </pre>	

	POP EBP RET
--	-----------------------------------